



```
In [57]: import pandas as pd
        from sklearn.model_selection import train_test_split
        from sklearn.datasets import load_iris
```

```
In [58]: df= load_iris()
```

```
In [59]: dir(df)
```

```
Out[59]: ['DESCR',
          'data',
          'data_module',
          'feature_names',
          'filename',
          'frame',
          'target',
          'target_names']
```

```
In [60]: df2= pd.DataFrame(df.data, columns=df.feature_names)
```

```
In [61]: df2
```

```
Out[61]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
In [62]: df2["target"]= df.target
```

```
In [63]: df2
```

Out[63]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

```
In [64]: x= df2.drop("target", axis=1)
         y= df2.target
```

```
In [65]: x_train, x_test, y_train, y_test= train_test_split(x,y, random_state=42, test_
```

```
In [66]: from sklearn.ensemble import RandomForestClassifier
```

```
In [67]: model= RandomForestClassifier(n_estimators=50)
```

```
In [68]: model.fit(x_train, y_train)
```

Out[68]:

▼ RandomForestClassifier ⓘ ?		
Parameters		
 <code>n_estimators</code>		50
 <code>criterion</code>		'gini'
 <code>max_depth</code>		None
 <code>min_samples_split</code>		2
 <code>min_samples_leaf</code>		1
 <code>min_weight_fraction_leaf</code>		0.0
 <code>max_features</code>		'sqrt'
 <code>max_leaf_nodes</code>		None
 <code>min_impurity_decrease</code>		0.0
 <code>bootstrap</code>		True
 <code>oob_score</code>		False
 <code>n_jobs</code>		None
 <code>random_state</code>		None
 <code>verbose</code>		0
 <code>warm_start</code>		False
 <code>class_weight</code>		None
 <code>ccp_alpha</code>		0.0
 <code>max_samples</code>		None
 <code>monotonic_cst</code>		None

```
In [69]: model.score(x_test, y_test)
```

Out[69]: 1.0

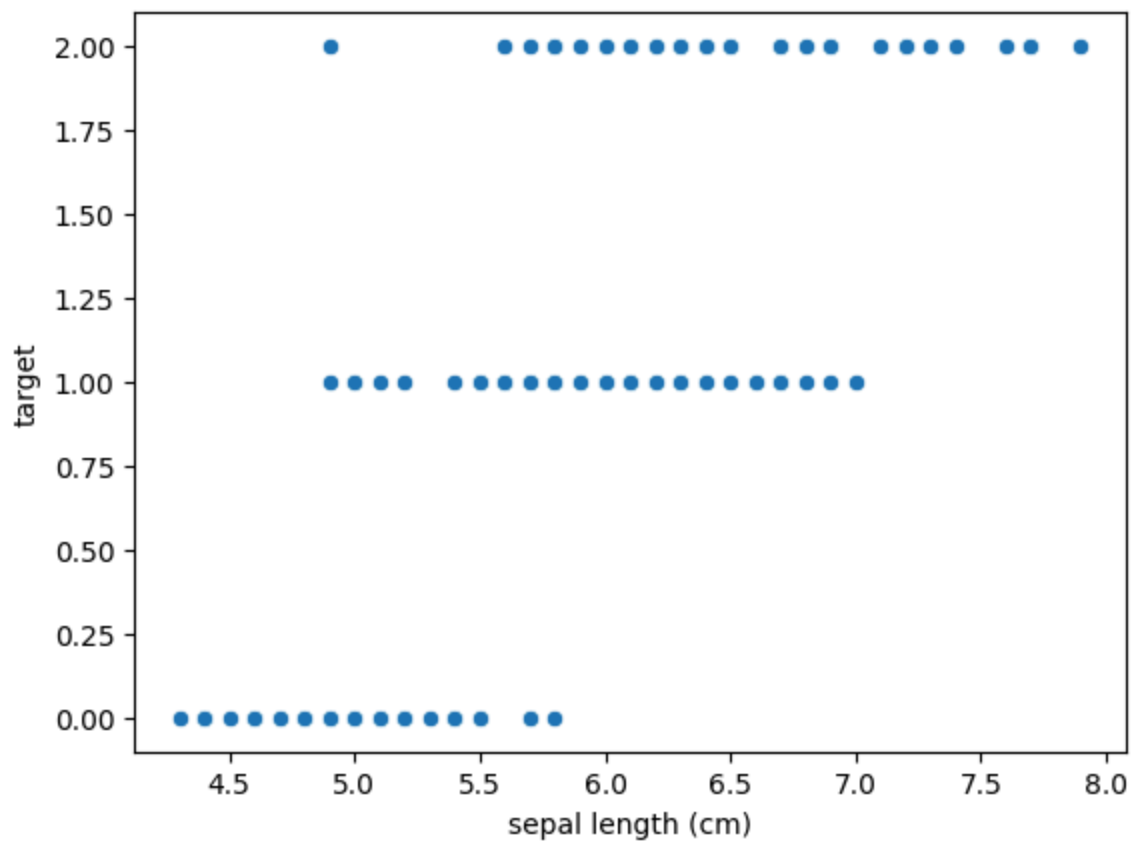
```
In [70]: model.score(x_train, y_train)
```

Out[70]: 1.0

```
In [ ]: import seaborn as sns
```

```
In [ ]: sns.scatterplot(x=df2['sepal length (cm)'], y=df2.target)
```

Out[]: <Axes: xlabel='sepal length (cm)', ylabel='target'>

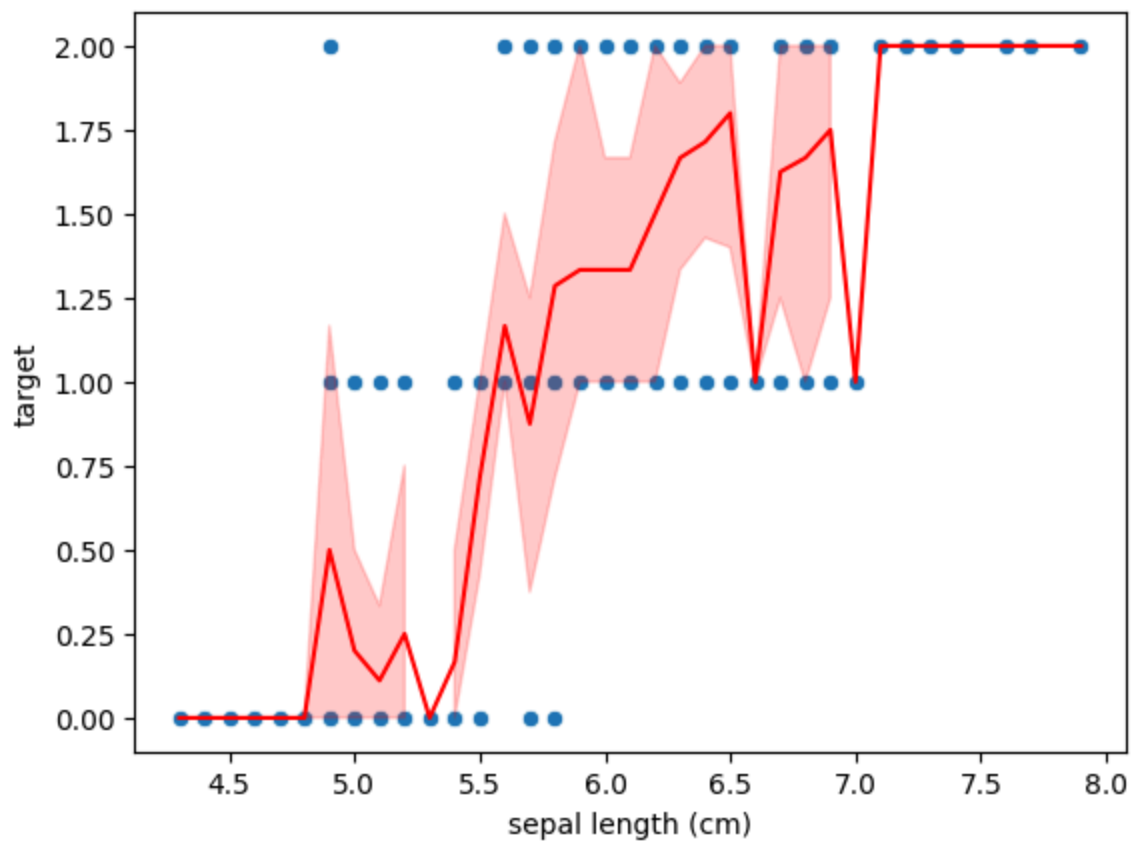


```
In [76]: df3= df2.sort_values("sepal length (cm)")
```

```
In [78]: y_pred=model.predict(df3.drop('target', axis=1))
```

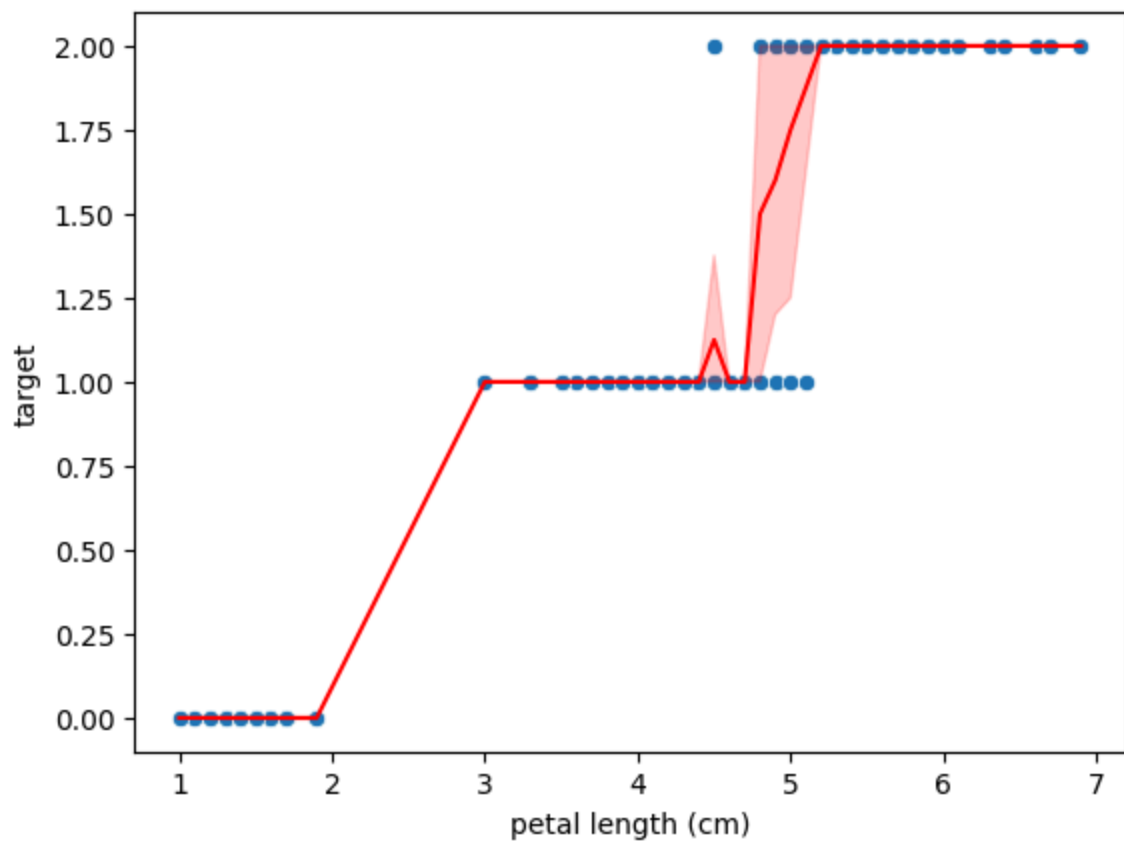
```
In [84]: sns.scatterplot(x=df2['sepal length (cm)'],y= df2.target)  
sns.lineplot(x=df3['sepal length (cm)'], y= y_pred, color="red")
```

```
Out[84]: <Axes: xlabel='sepal length (cm)', ylabel='target'>
```



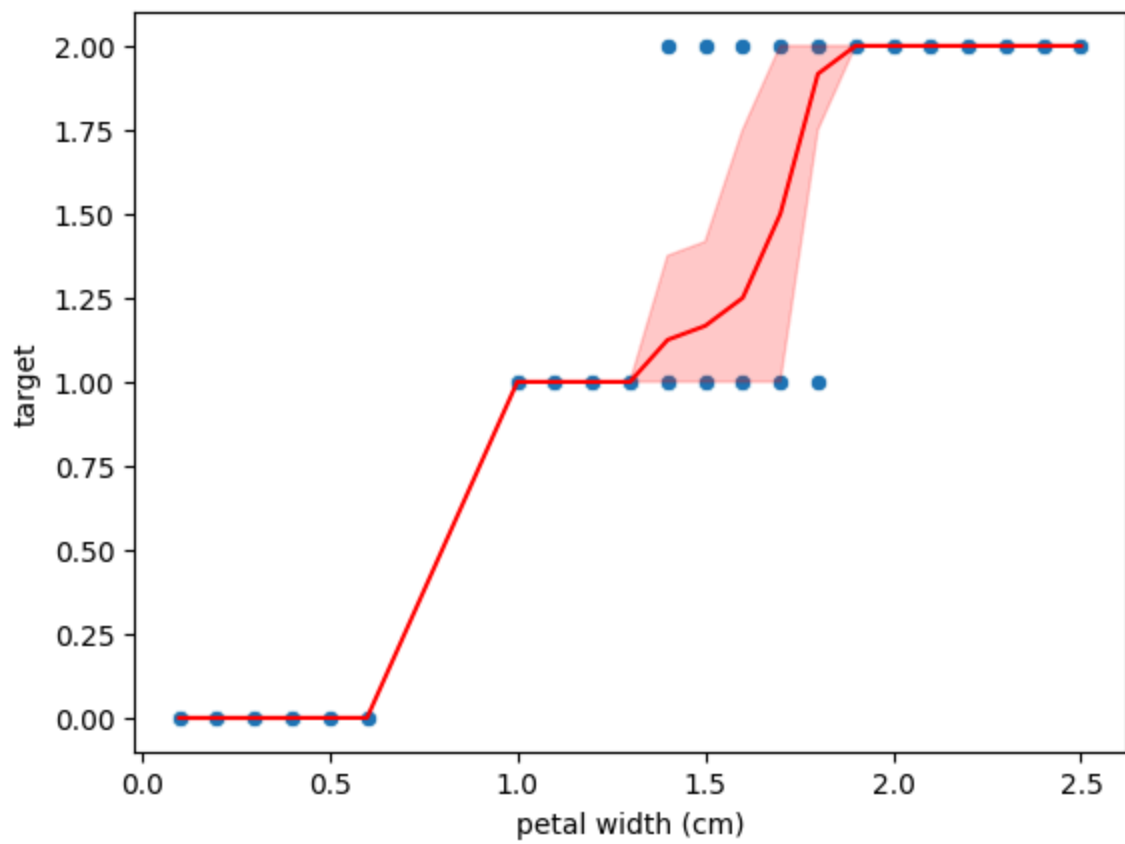
```
In [85]: sns.scatterplot(x=df2['petal length (cm)'],y= df2.target)  
sns.lineplot(x=df3['petal length (cm)'], y= y_pred, color="red")
```

```
Out[85]: <Axes: xlabel='petal length (cm)', ylabel='target'>
```



```
In [86]: sns.scatterplot(x=df2['petal width (cm)'],y= df2.target)
sns.lineplot(x=df3['petal width (cm)'], y= y_pred, color="red")
```

```
Out[86]: <Axes: xlabel='petal width (cm)', ylabel='target'>
```



In []: